Repaso Examen Práctico — Herencia, Clases Abstractas e Interfaces

1. Herencia (`extends`)

```
// SUPERCLASE
public class Vehiculo {
    protected String marca;   // protected = accesible en subclases
    protected int anio;

    public Vehiculo(String marca, int anio) {
        this.marca = marca;
        this.anio = anio;
    }

    public String descripcion() {
        return marca + " (" + anio + ")";
    }
}

// SUBCLASE
public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int anio, int puertas) {
        super(marca, anio);       // 1º: llamar al constructor del padre
        this.puertas = puertas;   // 2º: inicializar atributos propios
    }

    @Override
    public String descripcion() {
        return super.descripcion() + " - Puertas: " + puertas;
    }
}
```

Reglas a no olvidar:

- `super(...)` debe ser siempre la **primera línea** del constructor hijo.
- Atributos `private` del padre → no accesibles directamente, usar getters.
- Atributos `protected` del padre → accesibles directamente en la subclase.
- `@Override` + firma incorrecta → el compilador lo detecta como error.

2. Clase Abstracta (`abstract class`)

```
// CLASE ABSTRACTA (no se puede instanciar)
public abstract class FiguraGeometrica {
    protected String nombre;
    protected String color;

    public FiguraGeometrica(String nombre, String color) {
        this.nombre = nombre;
        this.color = color;
    }

    // Método concreto: heredado tal cual por las subclases
    public void mostrarInfo() {
        System.out.println(nombre + " | Color: " + color);
    }

    // Métodos abstractos: OBLIGATORIO implementar en subclases concretas
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
    public abstract void dibujar();
}

// SUBCLASE CONCRETA
public class Rectangulo extends FiguraGeometrica {
    protected double base;
    protected double altura;

    public Rectangulo(String nombre, String color, double base, double altura) {
        super(nombre, color);
        this.base = base;
```

```
        this.altura = altura;
    }

    @Override
    public double calcularArea() { return base * altura; }

    @Override
    public double calcularPerimetro() { return 2 * (base + altura); }

    @Override
    public void dibujar() {
        System.out.println("Dibujando rectángulo " + base + "x" + altura);
    }
}

// SUBCLASE DE SUBCLASE (herencia multinivel)
public class Cuadrado extends Rectangulo {
    public Cuadrado(String nombre, String color, double lado) {
        super(nombre, color, lado, lado);  // base y altura iguales
    }
    // No necesita @Override si la fórmula ya es correcta heredada
}
```

Reglas a no olvidar:

- `new FiguraGeometrica(...)` → **error de compilación**.
- Si una subclase no implementa todos los abstractos → debe ser `abstract` también.
- Método abstracto termina en `;`, sin llaves `{}`.

---

3. Interfaz (`interface` / `implements`)

```
// INTERFAZ (contrato puro)
public interface Empleado {
    // Todos los métodos son public abstract por defecto
    double calcularSalarioMensual();
    void mostrarDetalles();

    // Método default (Java 8+): tiene implementación, se puede sobrescribir o no
    default void imprimirRecibo() {
        System.out.println("Salario: " + calcularSalarioMensual());
    }
}

// CLASE CONCRETA que implementa la interfaz
public class EmpleadoRegular implements Empleado {
    private String nombre;
    private double salarioBase;

    public EmpleadoRegular(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    @Override
    public double calcularSalarioMensual() { return salarioBase; }

    @Override
    public void mostrarDetalles() {
        System.out.println("Empleado: " + nombre);
    }
}
```

---

4. Combinación: Clase Abstracta + Interfaz

El patrón más habitual en examen: interfaz → clase abstracta → subclases concretas.

```
// INTERFAZ
public interface Empleado {
    double calcularSalarioMensual();
    void mostrarDetalles();
}

// CLASE ABSTRACTA: implementa parte de la interfaz
public abstract class EmpleadoBase implements Empleado {
```

```
    protected String nombre;
    protected int id;
    protected double salarioBase;

    public EmpleadoBase(String nombre, int id, double salarioBase) {
        this.nombre = nombre;
        this.id = id;
        this.salarioBase = salarioBase;
    }

    @Override
    public void mostrarDetalles() {   // implementa este método de la interfaz
        System.out.println("Nombre: " + nombre + " | ID: " + id);
    }
    // calcularSalarioMensual() queda pendiente → cada subclase lo resuelve
}

// SUBCLASE CONCRETA A
public class EmpleadoRegular extends EmpleadoBase {
    public EmpleadoRegular(String nombre, int id, double salarioBase) {
        super(nombre, id, salarioBase);
    }

    @Override
    public double calcularSalarioMensual() { return salarioBase; }
}

// SUBCLASE CONCRETA B (con atributo extra)
public class Gerente extends EmpleadoBase {
    private double bonoAnual;

    public Gerente(String nombre, int id, double salarioBase, double bonoAnual) {
        super(nombre, id, salarioBase);
        this.bonoAnual = bonoAnual;
    }

    @Override
    public double calcularSalarioMensual() { return salarioBase + bonoAnual; }
}
```

---

5. Polimorfismo en el `main`

```
public class Main {
    public static void main(String[] args) {

        // Con clase abstracta
        FiguraGeometrica[] figuras = {
            new Rectangulo("r1", "azul", 4, 3),
            new Cuadrado("c1", "rojo", 5)
        };
        for (FiguraGeometrica f : figuras) {
            f.mostrarInfo();
            System.out.println("Área: " + f.calcularArea());
        }

        // Con interfaz
        Empleado[] empleados = {
            new EmpleadoRegular("Ana", 1, 1500),
            new Gerente("Luis", 2, 2000, 800)
        };
        for (Empleado e : empleados) {
            e.mostrarDetalles();
            System.out.println("Salario: " + e.calcularSalarioMensual());
        }
    }
}
```